

Challenges in Applying Hierarchical Reinforcement Learning in Atari Environments

Aidan Jared
Master of Data Science
ajar667@aucklanduni.ac.nz

Dominicus Nararya
Master of Data Science
dnar278@aucklanduni.ac.nz

Thomas Clunie
Master of Artificial Intelligence
tclu830@aucklanduni.ac.nz

Yuanyuan Yu
Master of Data Science
yyu119@aucklanduni.ac.nz

Jason Kong
Postgraduate Diploma in Computer
Science
jkon763@aucklanduni.ac.nz

Daoyu Zhang
Master of Data Science
dzha977@aucklanduni.ac.nz

Abstract—Hierarchical Reinforcement Learning has demonstrated superior performance in prior studies compared to Deep Q-Networks (DQN) in sparse reward environments such as Atari games. To evaluate these claims, we trained both a DQN and a Feudal Network in sparse Atari Environments. These models were then compared using *rlible* to generate statistically robust performance metrics. After training for 1×10^7 steps in *Ms. Pac-Man* we find that Feudal Networks outperform DQN. By generating 95% confidence intervals on independent testing runs, we find that the Feudal Network achieves a normalized IQM of 0.1787 [95% CI: 0.1673, 0.1901] compared to the DQN’s 0.1057 [95% CI: 0.0979, 0.1202]. The clear separation in performance indicates that Feudal Networks have higher performance in the *Ms. Pac-Man* environment. Despite these findings, Hierarchical Reinforcement Learning requires careful tuning of inputs, architecture, and hyperparameters to get the reported strong performance. Specifically, Feudal Networks can have unstable training without careful optimization and tuning of the environment. While we demonstrate that it outperforms DQN, it has significant flaws that make it challenging to apply in a random environment without extensive customization. Further research is needed to stabilize this architecture and facilitate broader adoption in practical applications.

Index Terms—Hierarchical Reinforcement Learning, Atari, Q-Learning, DQN, Double Q-Learning, Feudal Networks, *rlible*, attention, PyTorch

I. INTRODUCTION

Standard reinforcement learning (RL) models often struggle to learn desired policies in sparse reward environments. The limited performance feedback and lengthy action sequences necessitate models to develop long-term planning and reward assignment capabilities. In reinforcement learning, agents learn to select actions that maximize the cumulative sum of future rewards.

Sparse reward environments provide limited feedback to inform the model that it is progressing towards the goal. Hierarchical reinforcement learning aims to solve these issues by breaking the problem down into short-term and long-term components. Feudal Networks [1] have a Manager examine the long-term sequence of events and produce goals for the Worker, which focuses on immediate action selection. By breaking down the problem into separate components, Hierarchical

Reinforcement Learning has promise in learning in sparse reward environments.

The *Arcade Learning Environment (ALE)* [2] provides multiple training benchmarks. The games have sparse reward environments, which provide a challenge for standard Reinforcement Learning. *Ms. Pac-Man* provides an evolving environment where, as pellets are consumed, remaining rewards become sparser. Introducing a need for the model to explore the entire state space without receiving rewards for previously chosen routes [3]. Power pellets introduce resource management challenges that the model must learn to exploit for improved score and survivability. These challenges make *Ms. Pac-Man* an ideal test bed for examining how effectively Feudal Networks can learn in complex environments.

Feudal Networks introduce several training instabilities that can lead to cascading issues, preventing the model from learning optimal strategies. Challenges in training Feudal Networks include goals collapsing to singular vectors, Workers falling into local minima, and Workers learning to act contrary to the Manager’s goals.

We hypothesize that Feudal Networks will achieve a higher Inter-Quartile Mean score on *Ms. Pac-Man* than Deep Q-Networks after 10 million training steps. To test this, we implement both a Feudal Network and a Deep Q-Network baseline, training both to play *Ms. Pac-Man* and evaluating performance using the Inter-Quartile Mean from *rlible* [4].

Section II reviews related work in Reinforcement Learning with emphasis on sparse reward environments. Section III describes our experimental methodology and implementation. Section IV presents and analyzes our results. Section V concludes with a discussion of limitations and future work.

II. RELATED WORK

The development of Q-learning by Watkins [5] introduced the use of the Markov Decision Process to train models to learn the optimal policy π based on the State S , actions A , Probability P , and reward R to maximize the cumulative reward. This was extended to Deep Q-Learning [6] and Double Q-Learning [7], which leverage deep neural networks and stabilize training. While Q-Learning has shown strong

performance, it only considers the current state, lacking explicit long-term memory mechanisms.

To improve upon Q-Learning, Hierarchical Reinforcement Learning was developed. Dividing complex environments into different hierarchies has demonstrated strong performance in areas of sparse reward environments and long-term planning. Feudal Networks [1] splits problems into long-term and short-term sections with sub-models focusing on one aspect. The Manager operates at a lower temporal resolution, predicting state changes over the next c steps. The Worker produces actions based on its own perception of the state and the Manager’s goals.

Random Network Distillation (RND) [8] addresses sparse rewards by providing intrinsic motivation through state novelty: novel states yield high rewards while frequently-visited states provide minimal reward.

Gymnasium [9] and the integrated *ALE* [2] provide baseline environments for Reinforcement Learning Research. *ALE* uses an Atari emulator which hosts challenging games for models to learn, such as *Ms. Pac-Man* and *Montezuma’s Revenge*, which have become the benchmarks for testing Reinforcement Learning models in sparse reward environments.

High variance in Reinforcement Learning makes standard performance metrics statistically unreliable, complicating hypothesis testing. *rlible* [4] resolves these issues by generating confidence intervals and giving metrics that are robust to high variance.

III. METHODOLOGY

This study examines whether Hierarchical Reinforcement Learning outperforms Q-Learning in sparse reward environments. *Ms. Pac-Man* offers an evolving environment with initially dense rewards that become sparser as the game progresses. This provides a fair test bed for comparing Q-Learning and Feudal Networks. Early environmental rewards provide directional feedback while still requiring strategic planning to avoid ghosts and maximize score. Our implementation is available on GitHub¹

A. Design

To ensure a fair comparison, both models were trained under identical conditions. The *Ms. Pac-Man* environment provided by the *ALE* [2], accessed through the *Gymnasium* API [9] was used. Each model underwent 10 million training steps with equivalent input preprocessing, reward scaling, and an initial random seed to ensure experimental consistency. This setup aimed to isolate the effect of hierarchical architecture on performance in sparse reward scenarios while minimizing confounding variables.

The central hypothesis of this study is that the hierarchical structure of Feudal Networks, which comprises a long-term Manager and a short-term Worker, can enable more effective learning and exploration in sparse reward environments than the baseline DQN. Formally, the hypotheses are defined as follows:

- H_0 : Feudal Networks and DQN have equivalent performance on *Ms. Pac-Man* as measured by Inter-Quartile Mean, Performance Profiles, and Optimality Gap.
- H_1 : Feudal Networks achieves statistically higher performance than the DQN on *Ms. Pac-Man*, as measured by Inter-Quartile Mean, Performance Profiles, and Optimality Gap.

To provide a statistically sound assessment of model performance, evaluation was conducted using the *rlible* library [4]. This toolkit calculates robust statistical measures, including the Inter-Quartile Mean (IQM), Performance Profiles, and Optimality Gap. These metrics reduce sensitivity to outliers and employ bootstrap confidence intervals derived from multiple independent runs. This allows for a more reliable comparison of reinforcement learning models.

B. Environment

A standardized environment is required to test the hypothesis. The *Gymnasium* API [9] and *Ms. Pac-Man* in *ALE* [2] were used to train the models. To ensure equivalent training conditions, both the DQN and Feudal Network were trained on the 4 stacked greyscale frames of *Ms. Pac-Man* which was passed through *Gymnasium*’s Atari Preprocessing Module, which scaled down the image to 84×84 , and skipped the first 60 actions after termination to remove noise from training. Each action would be repeated for the subsequent 4 frames; these skipped frames would become the future State input to provide temporal information to the models. See Fig. 1. *Ms. Pac-Man* contains 9 actions: no-op, four cardinal directions, and four diagonal directions. The period between game start and game over is called an episode. Upon losing all lives, the episode would be terminated, and information would be masked from subsequent episodes.

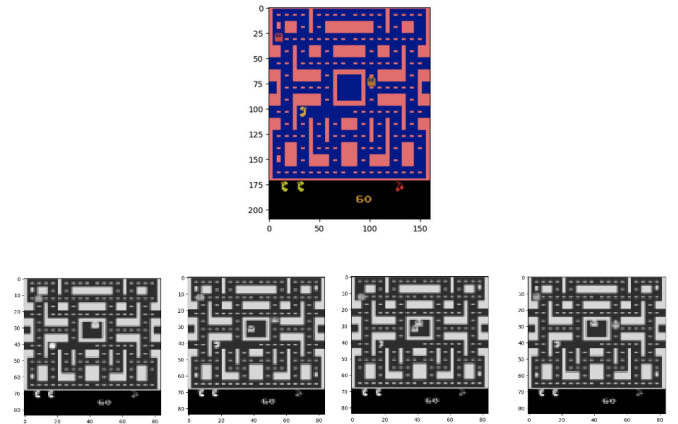


Fig. 1. Original input from *Gymnasium* and pre-processing results, which become the model inputs. Bottom 4 images are stacked as image channels and are the frames between steps.

ALE provides a standard mapping of indexes to actions across all emulated games, with index 0 being specified as a no-op or “do nothing this step”. This action type does cause issues with *Ms. Pac-Man* since the character retains the previous direction of motion, so no-op becomes continue. During

¹https://github.com/Aidan-Jared/COMPSCI_760_Reinforcement

training, models can learn that no-op doesn't stop movement, causing them to repeatedly select no-op, which prevents model exploration and requires the model to be trained from scratch.

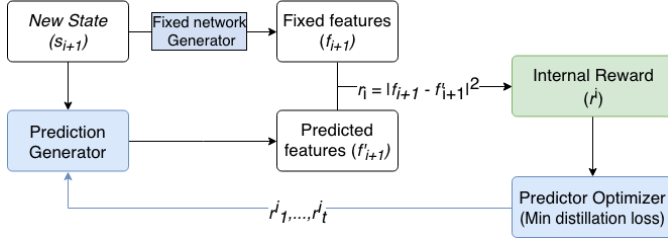


Fig. 2. RND model Architecture and how it provides rewards for novel States, image from [10] to illustrate how RND generates rewards for States. The State is run through the target and prediction network, and the reward is the MSE between the two predictions. The prediction network is trained alongside the RL model while the target has its weights frozen.

a) *Rewards*: Each *ALE* environment defines its own reward structure. To ensure comparable models, both models were trained using the same reward structure and individually optimized. To prevent unstable training dynamics, rewards were scaled down. See Table I for details. The only penalty was a custom addition for losing a life at -5 .

TABLE I
Ms. PAC-MAN REWARD SCALING

Reward Type	Original Reward	Scaled Reward
Pellets	10	1
Power Pellets	50	5
Fruit	100	10
Ghosts	200 – 1600	13.3 – 40
Death	0	-5

To improve model exploration in sparse reward environments, Random Network Distillation (RND) [8] was used to reward the discovery of novel States. A Target network has its weights frozen, and a Predictor is trained to match the Target. This is then used to reward the model with the Mean Squared Error between the Target and the Predictor. Fig. 2 illustrates RND architecture and implementation in training. More unique States provide higher rewards, while frequently seen States produce little to no reward. In environments such as *Montezuma's Revenge*, where rewards are infrequent, RND provides intrinsic motivation. To allow the model to learn basic gameplay and prevent destabilizing reward spikes, RND training occurs for 20 episodes before exploration rewards are provided.

The RND rewards are calculated through:

$$r_{\text{RND}} = \beta \times \text{MSE}_{\text{RND}} \quad (1)$$

where $\beta = 0.01$ for *Ms. Pac-Man* to provide an appropriate magnitude reward but not overwhelm other reward signals. RND was provided the RAM arrays from *Gymnasium* (a vector representation of the game's memory) at each step to produce a reward, and was trained alongside the main models with an identical training frequency and States. We chose to

use RAM instead of gameplay frames to provide RND with the true state information and to reduce model complexity. To ensure stability for the Feudal Network, the Intrinsic Reward was only given to the Worker.

C. Deep Q-Network

DQN provides a proven model architecture that can be inserted into a designed environment and trained to outperform a random agent.

a) *Architecture*: A Q-Model, as introduced by Watkins [5], learns to predict the cumulative reward for taking each action in a given state. As defined by equation (2).

$$R_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k}) \quad (2)$$

Where γ^k is the discount factor between $[0, 1]$ [5]. This was expanded with research into Deep Q-Networks (DQN) [6], which employs a Neural Network to learn the Q-Value function $Q(a, s; \theta)$, where θ is the network parameters. DQN learns by creating a frozen copy of the model to generate Target Q-Values, where loss is calculated by:

$$L(\theta) = E[r + \gamma \max(Q(a, s; \theta^-)) - Q(a, s; \theta)] \quad (3)$$

Where $Q(a, s; \theta^-)$ is the target's max prediction of the next State's Q-value, to maintain predictive ability, the target is periodically updated with the model's weights. This requires DQN to be trained off-policy, where every few steps, a replay buffer of previous States, actions, and rewards is randomly sampled from to produce a batch for the model to train on.

To stabilize and improve training, Double Q-Learning was implemented as per research done by Hado van Hasselt, Arthur Guez, and David Silver [7]. This changes the loss function so that the Model Q evaluates actions while the target Q' selects actions. Allowing for unbiased Q-value estimates to reduce maximization bias, which has been an issue in standard DQN learning.

b) *Training Configuration*: To train on *Ms. Pac-Man*, the DQN utilized two Convolutional Networks to process the state's frames and then used standard linear layers to predict the Q-Values, see Fig. 3. Every four steps, a batch of 32 was sampled from the previous 200,000 States to train on. This buffer size compensates for computational constraints while maintaining a robust sample of historical states. To calculate the loss, Smooth L1 loss was used to find the difference between the predicted Q and the target Q. After 1×10^7 steps, the model was saved and tested on 100 independent games of *Ms. Pac-man*. Table II contains more details on the used hyperparameters. To encourage exploration, there was a decreasing chance of selecting a random action to take instead of epsilon greedy. Starting at a 75% then decaying by a factor of .99985 every 1,280 actions. Was trained using an NVIDIA 4050 and Intel i9-14900HX with 32 GB of RAM.

D. Feudal

While DQN can learn basic actions in Atari environments, it underperforms a basic human player. In the original paper [1], Feudal Networks demonstrate higher performance than DQN in the same environment.

TABLE II
HYPERPARAMETERS FOR DQN AND FEUDAL NETWORK USED IN THIS PAPER

DQN Hyperparameters	Value	Feudal Hyperparameters	Value
Number of Environments	8	Number of Environments	8
Hidden Dim	512	Hidden Dim Manager	256
Batch Size	32	Hidden Dim Worker	16
Training Frequency	every 4 steps	Training Frequency	every 256 actions
Learning Rate	1×10^{-4}	Number Environments	8
Optimizer	Adam	Learning Rate	1×10^{-3}
Discount Factor (γ)	0.99	Optimizer	Adam
Total Training Steps	1×10^7	Discount Factor Manager γ_m	0.99
Target Update	10,000 steps	Discount Factor Worker γ_w	0.95
Replay Buffer Size	200,000	Total Training Steps	1×10^7
Random Exploration ε	0.75 to 0.1 decaying by 0.9985	Time Horizon c	10
RND delay	20 episodes	Dilation r	10
RND discount	0.01	Intrinsic Reward Coef α	0.1
		Entropy Coef ω	0.01
		Random Exploration ε	0.75 to 0.1 decaying by 0.9985
		RND delay	20 episodes
		RND discount	.01

a) *Architecture*: Feudal Networks extend the Actor-Critic architecture [11], which separates the policy from the value function estimation. DQN learns to predict future reward values for each action, using epsilon-greedy to select the action with the maximum expected value. Where epsilon greedy chooses the maximum value action. Actor-Critic methods use an actor that generates an action distribution for sampling, and a critic that estimates the value function. The loss calculation combines the critic’s advantage estimate with the log probability and entropy of the action distribution.

Feudal Networks utilizes a Manager to provide long-term goals to a Worker who acts upon the environment. The Manager is trained to produce goals that match the State change over the time horizon c :

$$\begin{aligned} \nabla g_t &= A_t^M \nabla_{\theta} d_{\cos}(s_{t+c} - s_t, g_t(\theta)) \\ A_t^M &= R_t - V_t^M(x_t, \theta) \end{aligned} \quad (4)$$

A_t^M is the difference between the reward and the Manager’s critic at t . The Worker receives rewards for achieving the goals, as well as for taking environmental actions.

To enable the Manager to operate at a lower temporal resolution for producing long-term plans, a Dilated LSTM (dLSTM) is used instead of an LSTM. The dLSTM splits its hidden State into r separate ‘cores’ which it attends to and updates every $t\%r$ steps ($\%$ is the modulo operation). This reduces the temporal resolution for the Manager while producing a long-term memory that an LSTM can use.

The Worker reads the goals as a sum of the goals over the time horizon and runs it through a network without bias, which produces weights to influence what action to take. To encourage exploration, random noise was added to the Workers’ action distribution at a chance of ε starting at 75% chance and decayed to 10%. This forces the model to take actions that it might not have taken, providing exploration in early training while not interfering with later model training. As shown in Fig. 4, the Feudal architecture as proposed in the original paper [1].

b) *Training Configuration*: The Feudal Network uses the same preprocessing and perception modules as DQN for envi-

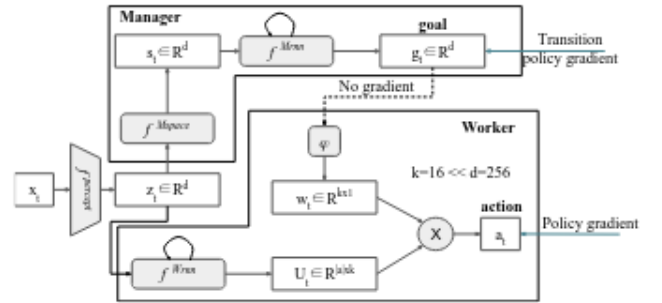
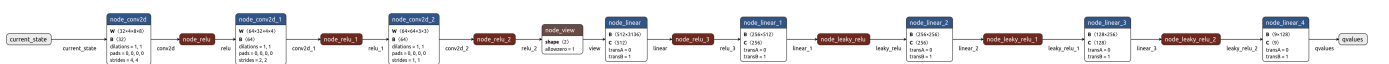


Fig. 4. A Feudal Network [1], CNN reads the input and sends spatial embedding to Manager and Worker, where Manager learns to predict State change and Worker learns to generate an action distribution. Both the Manager and Worker have a critic to predict the cumulative reward at each State.



ronment observation. Due to being designed as an expansion of the Actor-Critic architecture, Feudal Models are trained On-Policy. The model was trained directly on the previous 256 States in each Worker for a total batch size of 2048. The Manager focuses on long-term goals and thus requires a higher discount factor than the Worker ($\gamma_m = 0.99$ versus $\gamma_w = 0.95$). Additionally, the Manager requires extra parameters such as the Time Horizon $c = 10$ and dilation $r = 10$, which were chosen to replicate Vezhnevets' implementation from scratch [1]. This also reduced the total variance in States, which allows the Manager to generate accurate goals. The hidden dimension of 256 was also from the original paper, and it was used to keep the computational load balanced.

The Worker uses a smaller hidden dimension of 16 to encode action-specific information efficiently and maintain manageable goal representations. This was once again based upon the original paper, while the n_{actions} is dependent on the training environment, which is 9 for *Ms. Pac-Man*. The Worker was given $r_i = \alpha(\nabla_{\theta} d_{\text{cos}} + 0.01r_{\text{RND}})$ for the following goals and exploring the environment. Where $\nabla_{\theta} d_{\text{cos}}$ is the similarity between the Manager's goals and the state change over the time horizon. This intrinsic reward is scaled down by $\alpha = 0.1$ to not oversaturate rewards. To encourage early exploration, random noise is injected into the action distribution with 75% probability, which is then sampled accordingly. This exploration rate decays by a factor of .9985 per update to a minimum of 10%. This approach ensures early exploration while preserving informative action distributions during evaluation.

The same optimizer was used for Feudal as DQN, see Table II for hyperparameters comparison. A custom loss calculation was used for back propagation [1]:

$$\begin{aligned} \nabla g_t &= A_t^M \nabla_{\theta} d_{\text{cos}}(s_{t+c} - s_t, g_t(\theta)) \\ A_t^M &= R_t - V_t^M(x_t, \theta) \end{aligned} \quad (5)$$

The Manager's loss calculation.

$$\begin{aligned} A_t^W &= \left(R_t + \alpha \left(\nabla_{\theta} d_{\text{cos}} + \frac{r_{\text{RND}}}{100} \right) \right) - V_t^W(x_t, \theta) \\ L_{\text{Worker}} &= A_t^W \log(p_{\text{action}}) \end{aligned} \quad (6)$$

The Workers' Loss calculation

$$\begin{aligned} L_{\text{value}_m} &= \frac{1}{10} (A_t^M)^2 \\ L_{\text{value}_w} &= \frac{1}{10} (A_t^W)^2 \end{aligned} \quad (7)$$

The Critic loss calculation

$$L_{\text{total}} = -L_{\text{Worker}} - \nabla g_t + L_{\text{value}_m} + L_{\text{value}_w} - \omega S \quad (8)$$

The total loss calculation, including the entropy of the action distribution S . Was trained using an NVIDIA 4050 and Intel i9-14900HX with 32 GB of RAM.

E. Limitations and Challenges

DQN's primary limitation is its large memory requirement. The replay buffer must store a large number of samples; storing 200,000 + states with associated metadata consumes substantial memory. A high computer RAM is required to

have a large enough replay buffer to sample from. To reduce the memory impact, game frames were saved as integer images of $[0, 255]$ in the buffer and rescaled back to $[0, 1]$ when sampled.

Standard DQN selects actions by maximizing predicted Q-values, which can lead to overestimation bias. This occasionally causes overestimation bias, which destabilizes training by compounding estimation errors. Double Q-Learning reduces overestimation bias by decoupling target and evaluation networks. Using double Q-learning helped stabilize the training of the DQN, which produced a better baseline model.

To effectively learn, Feudal Networks require a balance between the individual components and the State. The Manager's goals represent spatial differences between states, limiting the architecture's effectiveness to visual domains. There may be ways to train the Manager to produce goals not involving spatial differences, but that is out of scope.

The Manager maximizes cosine similarity $d_{\text{cos}}(s_{t+c} - s_t, g_t(\theta))$ while the Critic minimizes $R_t - V_t^M(x_t, \theta)$; these are two conflicting objectives that can destabilize training and produce suboptimal models. When $V_t^M \geq R_t$ and $d_{\text{cos}} < 0$, the loss gradient is attenuated, slowing Manager learning. These issues are compounded by the Worker receiving disincentives for following Manager goals, causing the two components to work at cross-purposes. A possible solution is to add a goal quality term:

$$g_q = \frac{1}{n} \sum ((s_{t+c} - s_t) - g_t(\theta))^2 \quad (9)$$

g_q trains the model to match the magnitude of the State change alongside the cosine similarity. However, this may cause the network to prioritize magnitude matching over directional alignment. Another possible solution is shifting the cosine similarity from $[-1, 1]$ to $[0, 1]$ through

$$d'_{\text{cos}} = \frac{d_{\text{cos}} + 1}{2} \quad (10)$$

However, shifting the cosine similarity range reduces penalties for poor alignment.

The log probabilities and entropy of the predictions are included in the loss calculation to encourage Worker exploration. No such exploration term exists for the Manager, which can cause it to enter a local minima and produce identical goals across all states. Adding a goal entropy term based on how similar all goals are can help, but it degrades d_{cos} . A balance is possible between these terms with careful tuning, improving exploration.

The Manager is highly sensitive to state normalization. Without normalization, states cause the Manager to produce goals with little semantic value for the Worker. To generate actions, the Worker sums the received goals alongside its hidden State to create the action distribution. When the State embedding is not normalized, the Manager starts producing goals centered on only a few features. This signal can overwhelm the Worker's hidden state, producing heavily skewed action distributions that reduce exploration. In worst-case scenarios, this gets tied to the Worker learning an optimal action, leading to a collapse of the action distribution to a single

prediction. To reduce the Worker from being overwhelmed, the Worker can be modified to

$$w_t = \varphi \left(\sum_{i=t-c}^t g_i \right) \quad (11)$$

$$w_t = \tanh(w_t) \quad (12)$$

Equations (11) and (12) show how the Worker processes goals through φ , summing over the previous c goals to produce action weights w_t .

$$h_t^W, U_t = f^{\text{Wnn}}(z_t, h_{t-1}^W) \quad (13)$$

$$\pi_t = \text{SoftMax} \left(\frac{U_t w_t}{\sqrt{k}} \right) \quad (14)$$

Equation (13) represents the Worker's LSTM, and Equation (14) combines the Worker's predictions with goal-derived weights to form the action distribution. Where k is the Worker's hidden dimension, this provides direction from the weights but prevents large values from overwhelming the Worker's prediction. This can prevent un-normalized inputs, leading to over-scaled goals.

Conversely, excessive normalization can suppress important features, preventing the model from learning salient environmental characteristics. There are two possible solutions to this issue, but both increase the number of parameters. The first is using stacked RGB images instead of greyscale, so the model has more spatial data alongside the temporal information that the stacking provides. Another possible solution is implementing an Attention Mechanism similar to a Convolutional Block Attention Module [12] to learn useful features across channels and spatial dimensions. Normalizing features across the batches does produce strong training performance but leads to weak test performance, indicating that batch normalization produces a signal only visible when running multiple batches, which does not reflect real-world situations.

Further hyperparameter tuning is also required to maximize the performance of the models and ensure their best performance. The computational limitations prevented testing larger models, longer time horizons, larger buffer sizes, and more complex CNNs. Future work should tune the hyperparameters to maximize the performance of both models.

F. *reliable*

Reinforcement learning exhibits high variability across different test runs, necessitating the use of unique metrics to evaluate model performance. The *reliable* package [4] uses bootstrapping on the performance data to generate confidence intervals while relying on more robust metrics to assess model performance. Mean and Median provide baseline performance measurements. However, the Mean is heavily influenced by extreme outliers, while the Median is more robust to low-performing runs. *reliable* used the IQM, Performance Profiles, and Optimality Gap to produce statistically sound evaluation metrics.

To generate metrics, both models were run on 100 independent test games of *Ms. Pac-Man*. The scores of these games were then bootstrapped to a sample of 5,000 by *reliable* to

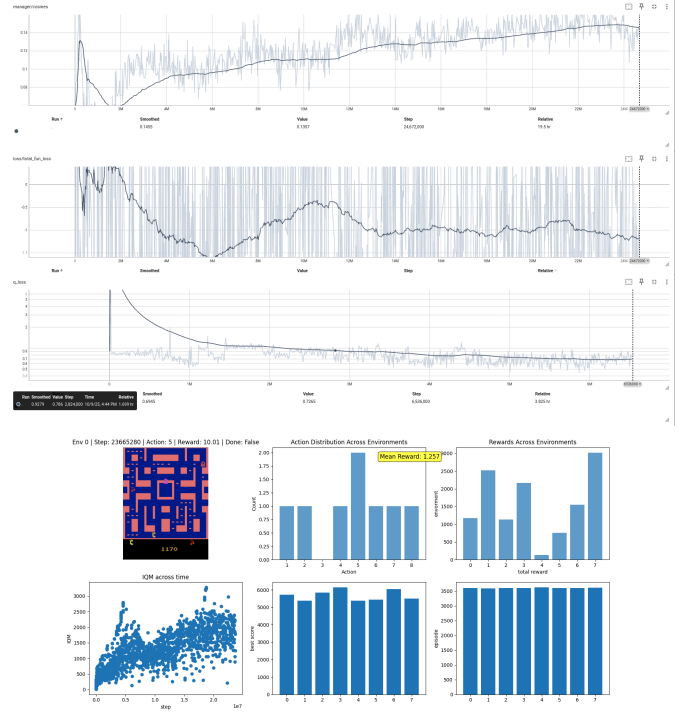


Fig. 5. Cosine similarity of Manager over 2×10^7 steps. Total loss of Feudal Network over 2×10^7 steps. Monitoring training status with Action distribution, reward distribution, IQM, best score, and current episode. Plots from model training in *TensorBoard*.

produce confidence intervals. Scores were then normalized using the following equation:

$$\tau = \frac{S - S_{\text{random}}}{S_{\text{human}} - S_{\text{random}}} \quad (15)$$

Where S is the score from each run, $S_{\text{random}} = 307.3$, and $S_{\text{human}} = 6952$ (values from original *reliable* implementation [4]).

These values were then used to calculate the Mean, Median, IQM, and Optimality Gap. The IQM is calculated by removing the top and bottom 25% of scores and computing the mean of the remainder. This provides a metric that is robust to outliers while reflecting the model's true typical performance. The Optimality Gap represents the percentage of test runs below an experimenter-defined threshold. For this paper, a threshold of $\tau = 0.1057$, the DQN IQM was used. This was chosen to directly compare the two models; if they have similar IQM, then the Optimality Gaps should be similar. However, one model may have more runs with a score greater than the IQM. This would reduce the Optimality Gap and be an indication of strong performance. See equation (16)

$$\text{OG} = \frac{|A < \tau|}{|A|} \quad (16)$$

Where $|A|$ is the number of samples. A perfect Optimality Gap equals 0.

IV. RESULTS

After training each model for 1×10^7 steps on *Ms. Pac-Man*, the Feudal Network demonstrated a statistically signif-

Comparison of Score Distributions on Ms. Pac-Man

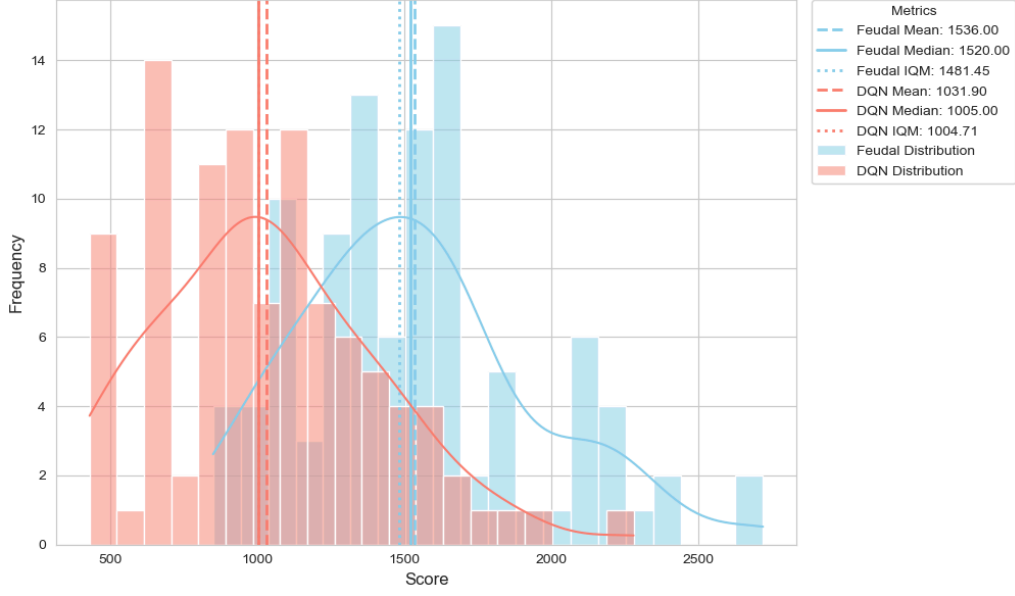


Fig. 6. Score Distribution of Feudal Network Model and Q model on *Ms. Pac-man*.

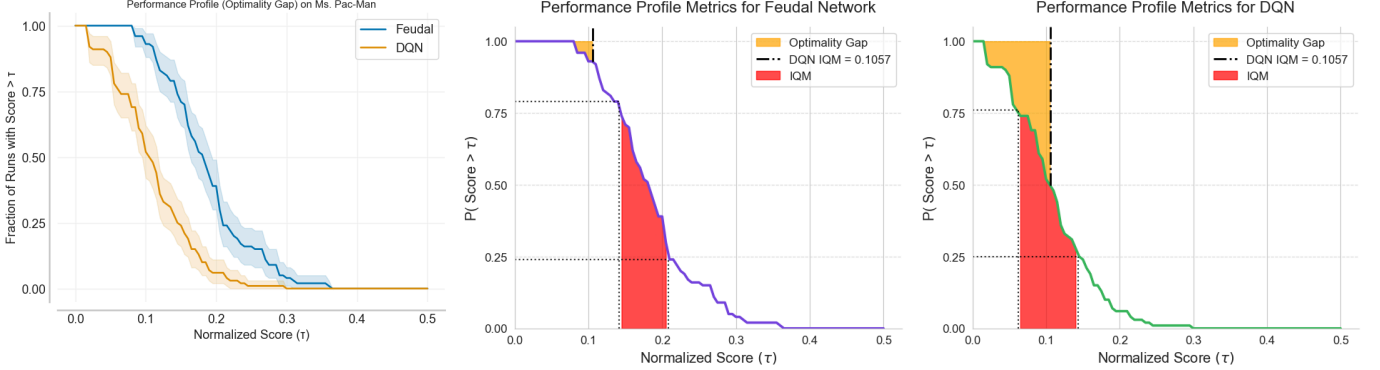


Fig. 7. Performance Profiles showing the percentage of runs at each normalized score τ , where $\tau = 0$ represents random agent performance and $\tau = 1$ represents human-level performance [4]. The Optimality Gap (orange area) represents runs below the threshold $\tau = 0.1057$; both models achieved 100% gap. Shaded regions on the left-most plot represent 95% confidence intervals.

icant performance advantage over the baseline DQN. The results support our alternative hypothesis (H_1), indicating that the hierarchical architecture is more effective for this task. This superiority is evident across aggregate statistics, score distributions, and probabilistic performance profiles.

A. Central Tendency and Score Distribution

A direct comparison of central tendency metrics Fig. 6 demonstrates statistically significant differences. The Feudal Network achieved an IQM of 1481.45 (.1794), a mean of 1536.00 (0.1848), and a median of 1520.00 (.1824). In contrast, the DQN agent’s performance was considerably lower, with an IQM of 1004.71 (.1027), a mean score of 1031.90 (0.1090), and a median of 1005.00 (.1049). The parentheses contain the normalized score. See Table III for 95% confidence intervals. This stark difference in performance is visually corroborated by the score distributions

shown in Fig. 6. The Feudal Network’s scores are tightly clustered in a high-scoring range, primarily between 1500 and 1750, indicating a highly consistent and reliable agent. The DQN agent’s distribution is substantially wider and multi-modal, signifying high variance and unpredictability. Although capable of occasional high scores, DQN also produced a large number of very low-scoring games, failing to develop a consistently effective strategy.

B. Probabilistic Performance Comparison

A more nuanced comparison is provided by the performance profiles generated using the *reliable* library, shown in Fig. 7. These profiles plot the probability of a model achieving a score greater than a given normalized threshold.

Both models exhibit steep drop-offs. Nevertheless, the Feudal Network’s profile consistently has a higher $P(\text{Score} > \tau)$ than DQN. We can use this to conclude that until $\tau =$

TABLE III
MODEL PERFORMANCE METRICS FOR NORMALIZED SCORES

Metric	DQN	DQN 95% CI	Feudal	Feudal 95% CI
IQM	0.1057	(0.0929, 0.1179)	0.1787	(0.1673, 0.1901)
Median	0.1049	(0.0929, 0.1177)	0.1824	(0.1643, 0.1914)
Mean	0.1090	(0.0979, 0.1202)	0.1848	(0.1731, 0.1967)
Optimality Gap threshold $\tau = 0.1057$	0.0208	(0.0153, 0.0266)	0.0012	(0.0003, 0.0022)

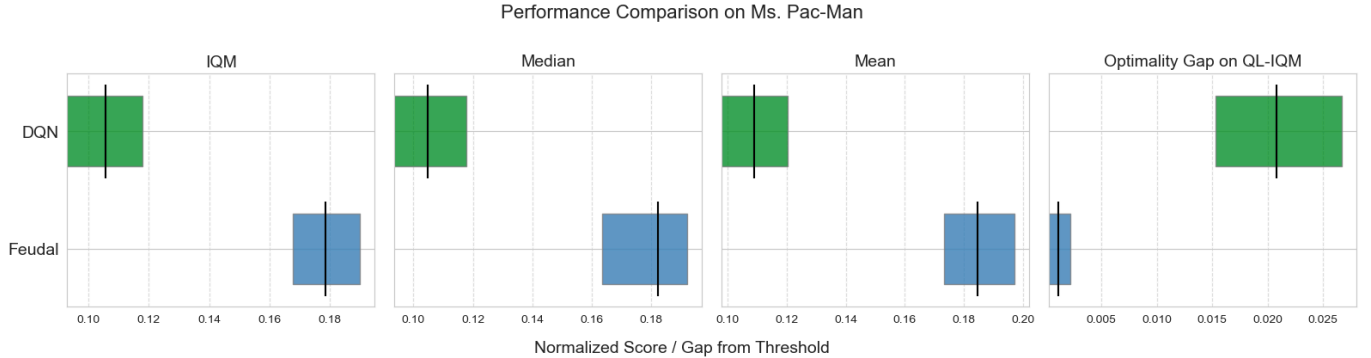


Fig. 8. IQM, Median, and Mean of Feudal Network & Q-learning on *Ms. Pac-man*. The metric has 95% confidence intervals around the performance as generated by bootstrapping the 100 test runs through *reliable*. The Feudal Network and DQN have no overlap in confidence intervals, indicating a 95% chance that Feudal outperforms DQN. Similarly, the Optimality Gap has a smaller percentage of Feudal runs performing below the DQN IQM, indicating a better performance.

.38, the Feudal Network has a high probability of generating a higher score than DQN. The Optimality Gap (orange area), which represents the average performance deficit from a perfect score, is visually similar for both.

These empirical results demonstrate that the Feudal Network achieves both higher average scores and, more critically, significantly greater consistency than DQN. These findings confirm the rejection of the null hypothesis and highlight the practical benefits of the Feudal Network’s hierarchical approach in navigating the complex environment of *Ms. Pac-Man*.

The more telling metric, and the focus of our hypothesis, is the IQM, represented by the red shaded area. The Feudal Network’s IQM is significantly higher, being shifted further right than the DQN’s IQM.

The aggregate performance comparison in Fig. 8 and Table III further solidifies these findings, showing higher IQM and median scores for the Feudal Network with tight confidence intervals, confirming the statistical significance of its advantage.

C. Statistical Analysis

All reported metrics, including the IQM, Median, and Mean, were computed using the 5000-sample bootstrap procedure implemented in *reliable*, based on 100 independent evaluation runs for each model.

The bootstrap procedure is equivalent to a hypothesis-testing process, because it fulfills the basic components of a statistical test: it defines a null hypothesis (no true difference between models), computes the test statistic (the IQM difference), and estimates its random variation through resampling.

In practice, when the bootstrap confidence intervals for two models do not overlap, it already means that their difference is significant enough that it is very unlikely to be caused by random chance.

As illustrated in Fig. 8, the Feudal Network consistently outperforms the DQN across all bootstrap resamples, and the 95% confidence intervals of their IQM values do not overlap, indicating that the observed performance difference is statistically significant. Therefore, we can say with 95% confidence that the Feudal Network’s higher IQM reflects a genuine and statistically significant improvement.

This result confirms that the hierarchical Feudal Network achieves higher scores on average and more stable and reliable learning performance compared to the baseline DQN in the sparse-reward *Ms. Pac-Man* environment.

V. CONCLUSION

In this paper, we examined the performance of Feudal Networks and DQN models in a sparse reward environment. We compared the models after training 1×10^7 steps on *Ms. Pac-Man* in *ALE* and using statistically sound metrics from *reliable*.

Our results show that after equivalent training conditions, the Feudal Network outperformed DQN, supporting our hypothesis. With a higher IQM, lower variance, and a lower Optimality Gap than DQN, the Feudal Network demonstrated statistically significant superior performance.

However, these performance gains come with significant implementation challenges that warrant further investigation. A key limitation is the instability of the Manager-Worker

interaction. The Feudal Network is highly sensitive to state normalization, reward scaling, and hyperparameter tuning, leading to goal collapse and contradictory gradients between the Manager and Worker. Future work should explore stabilizing the hierarchical relationship through several approaches. First, adaptive normalization techniques could dynamically balance Manager and Worker learning rates. Second, attention-based goal selection modules (e.g., CBAM [12]) could help the Manager identify salient spatial and temporal features. Third, multi-objective loss formulations could better align long-term and short-term optimization objectives.

Extended training beyond 1×10^7 steps may yield more stable convergence and improved performance for both architectures. Additionally, investigating non-spatial goal representations independent of CNN-derived features would enable the Feudal architecture to generalize beyond visual domains to symbolic reasoning tasks. Alternative intrinsic motivation mechanisms, such as curiosity-driven learning [10], may further enhance exploration efficiency beyond RND.

Integrating attention mechanisms into different model components could also lead to stronger performance. The perception modules could integrate CBAM [12] to enhance hierarchical coordination and temporal credit assignment. Similarly, updating how the Worker interprets goals with an attention mechanism could help the Worker attend to goals across the temporal horizon. This would enable selective goal weighting rather than simple summation over the time horizon. Further research into generating non-spatial goals is needed for Feudal Networks to be used in more practical situations.

More broadly, Hierarchical Reinforcement Learning should be validated in real-world continuous-control and multi-agent environments where sparse and delayed rewards are inherent. Such validation would demonstrate the robustness of hierarchical methods and bridge the gap between experimental research and practical deployment in complex decision-making systems.

VI. CONTRIBUTIONS

Aidan Jared was the group leader and worked on the API for *Balatro*, implementing the Feudal Network, Transformer model, training loop for both feudal and transformer models, as well as reward wrappers for *Ms. Pac-Man* and *Montezuma's Revenge*, testing loop, all utility code, player data collection script, and did some implementation of CBAM into the perception model. Debugged the DQN model for functionality and computational limitations, trained and tested all the models. Investigated different stabilization methods for the perception and the Manager. Also wrote the majority of presentations, wrote multiple sections of the literature review (see author contributions on literature review for breakdown of what each individual did), and edited the literature review. Wrote Section I, Section II, Section III, worked on Section V, and edited.

Dominicus worked on evaluating model performance using *rliaible*, expanded the reward wrapper for *Montezuma's Revenge*, generated all plots and metric data, added research

papers to the project proposal presentation, set the research question, and contributed to the literature review. Wrote Section III.F and Section IV.

Thomas² designed the DQN and its associated training and utility functions. Added information and diagrams to the model design on the project proposal presentation. Contributed to Literature Review.

Yuanyuan Yu wrote Section IV.C, worked on Section III.A, Section V, and contributed to the literature review.

Jason Kong worked on the literature review and did project presentations.

Daoyu Zhang did early data analysis for *Balatro* and did project presentations.

REFERENCES

- [1] A. S. Vezhnevets *et al.*, “FeUdal Networks for Hierarchical Reinforcement Learning,” *arXiv preprint arXiv:1703.01161*, 2017, [Online]. Available: <https://arxiv.org/abs/1703.01161>
- [2] M. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The Arcade Learning Environment: An Evaluation Platform for General Agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, Jun. 2013.
- [3] S. Greydanus, A. Koul, J. Dodge, and A. Fern, “Visualizing and Understanding Atari Agents,” *arXiv preprint arXiv:1711.00138*, 2018, [Online]. Available: <https://arxiv.org/abs/1711.00138>
- [4] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, “Deep Reinforcement Learning at the Edge of the Statistical Precipice,” *CoRR*, 2021, [Online]. Available: <https://arxiv.org/abs/2108.13264>
- [5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992, doi: 10.1007/BF00992698.
- [6] V. Mnih *et al.*, “Playing Atari with Deep Reinforcement Learning,” *CoRR*, 2013, [Online]. Available: <http://arxiv.org/abs/1312.5602>
- [7] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” *CoRR*, 2015, [Online]. Available: <http://arxiv.org/abs/1509.06461>
- [8] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by Random Network Distillation,” *arXiv preprint arXiv:1810.12894*, 2018, [Online]. Available: <https://arxiv.org/abs/1810.12894>
- [9] M. Towers *et al.*, “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [10] R. Horev, “Explained: Curiosity-Driven Learning in RL.” Accessed: Oct. 07, 2025. [Online]. Available: <https://medium.com/data-science/explained-curiosity-driven-learning-in-rl-exploration-by-random-network-distillation-72b18e69eb1b>
- [11] V. Mnih *et al.*, “Asynchronous Methods for Deep Reinforcement Learning,” *CoRR*, 2016, [Online]. Available: <http://arxiv.org/abs/1602.01783>
- [12] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” *CoRR*, 2018, [Online]. Available: <http://arxiv.org/abs/1807.06521>

²Had to leave the course, but still did large amounts of work to move the project forward